

1 Brief description

A stochastic cellular automata running a 1k-RAM, 30kHz 6502 CPU pseudo-asynchronously at each cell.

2 Introduction

A cellular automata where each cell runs a virtual 6502, a classic CPU released in 1975. The appeal to retro computing is designed to win a wide audience, to support which we will also develop mobile/serverless app infrastructure to allow mobile code to run on peoples' phones and hop between phones, with mutation and selection. A client viewer layer will allow visual interpretation of virtual memory, as well as local policies e.g. migration to/from server (and/or local neighborhood) or suppression of particular programs. The long-term goal is to obtain a dataset that will allow us to ask whether we can use the tools of population genetics and phylogenetics to study mobile code on a large scale. Subprojects will include the design of various agent-based and cellular automata models from physics, biology, chemistry, and other fields of science and A-Life. To motivate broad adoption we will offer prizes for the most creative and most virally successful programs.

- The paged storage is addressed as a square array of cells with $B = 256$ cells per side and $M = 1024$ bytes per cell, so there are $B^2 = 65,536$ (64k) cells requiring $B^2M = 64\text{Mb}$ of storage.
- The memory-mapped neighborhood is 7x7 cells around the local origin (0,0). Cell n occupies the space from $1024n$ bytes to $1024n + 1023$. The relationship between cell index, memory location, and offset (x,y) are shown in these tables

Memory pages.

	$x = -3$	$x = -2$	$x = -1$	$x = 0$	$x = 1$	$x = 2$	$x = 3$
$y = 3$	C0..C3	B0..B3	90..93	64..67	74..77	94..97	B4..B7
$y = 2$	AC..AF	60..63	50..53	24..27	34..37	54..57	98..9B
$y = 1$	8C..8F	4C..4F	20..23	04..07	14..17	38..3B	78..7B
$y = 0$	70..73	30..33	10..13	00..03	08..0B	28..2B	68..6B
$y = -1$	88..8B	48..4B	1C..1F	0C..0F	18..1B	3C..3F	7C..7F
$y = -2$	A8..AB	5C..5F	44..47	2C..2F	40..43	58..5B	9C..9F
$y = -3$	BC..BF	A4..A7	84..87	6C..6F	80..83	A0..A3	B8..BB

Each time execution resumes after an interrupt, the origin is resampled and the neighborhood rotated by a random multiple of 90 degrees. Zero page locations 0xF0-0xF9 are designated as "oriented registers", whose top 6 bits are rotated along with the memory map when a different orientation is sampled. (Note: 0xF9 is the storage location for PCHI, so this is also rotated.)

Cell numbers:

	$x = -3$	$x = -2$	$x = -1$	$x = 0$	$x = 1$	$x = 2$	$x = 3$
$y = 3$	48	44	36	25	29	37	45
$y = 2$	43	24	20	9	13	21	38
$y = 1$	35	19	8	1	5	14	30
$y = 0$	28	12	4	0	2	10	26
$y = -1$	34	18	7	3	6	15	31
$y = -2$	42	23	17	11	16	22	39
$y = -3$	47	41	33	27	32	40	46

Cell coordinates:

Range	Directions
0	Origin
1..4	$N=(0,+1)$, $E=(+1,0)$, S , W
5..8	NE , SE , SW , NW
9..12	N^2 , E^2 , S^2 , W^2
13..20	N^2E , NE^2 , SE^2 , S^2E , S^2W , SW^2 , NW^2 , N^2W
21..24	N^3 , E^3 , S^3 , W^3
25..27	N^2E^2 , S^2E^2 , S^2W^2 , N^2W^2
28..35	N^3E , NE^3 , SE^3 , S^3E , S^3W , SW^3 , NW^3 , N^3W
36..43	N^3E^2 , N^2E^3 , S^2E^3 , S^3E^2 , S^3W^2 , S^2W^3 , N^2W^3 , N^3W^2
44..48	N^3E^3 , S^3E^3 , S^3W^3 , N^3W^3

- The area from 0xE000-0xEE3F contains some read-only lookup tables for mapping various transformations inside the neighborhood (operations that yield vectors outside the neighborhood return 0xFF)

Table start S	Meaning of $S[i]$
0xE000 +64 <i>j</i>	Vector addition $v_i + v_j$
0xEC40	Rotation 90° clockwise
0xEC80	Rotation 180°
0xECC0	Rotation 270°
0xED00	Reflection about x-axis
0xED40	Reflection about y-axis
0xED80	X coordinate of v_i plus 3
0xEDC0	Y coordinate of v_i plus 3
0xEE00	See below

The table from 0xEE00-0xEE3F contains the mapping from (x, y) coordinates to cell indices, with y ascending fastest, starting from cell $(-3, -3)$; so the byte in $0xEE00 + (y + 3) + 64(x + 3)$ is the index of cell with relative offset (x, y) for $-3 \leq x, y \leq 3$.

- We aim to clock the entire board at the rate of a 1981 BBC Micro (2Mhz), so each cell updates at 30.5Hz. (This is a lower bound. There is no reason you can't run the board faster.)
- Interrupts arrive as an (approximately) Poisson process with an average rate of 1 per 4,096 cycles.

Interrupt model. Pre-emptive scheduling is conceptually an IRQ (which the 6502 program can mask using SEI/CLI) followed by an NMI (which it cannot). Memory writeback happens, if the program allows it, between the IRQ and the NMI. Setting the I flag (SEI) thus creates an atomic transaction: writes accumulate until a BRK commits them (software interrupt), and are reverted if a timer interrupt fires instead.

A timer interrupt is handled as follows:

- If the interrupt disable flag (I) is set, all writes made since the last interrupt are reverted (atomic abort). Otherwise:
 - The B flag (bit 4) is cleared in P.
 - CPU registers are written to the last seven bytes of zero page (0xF9–0xFF).
 - The memory-mapped neighborhood is copied from RAM to storage (un-rotating oriented registers at 0xF0–0xF9).
 - A new origin cell (i, j) and orientation is randomly sampled. The memory-mapped neighborhood of that cell is loaded from storage to RAM.
 - The last four bytes of zero page are overwritten with pseudorandom numbers.
 - CPU registers are restored from the last seven bytes of zero page.
- A BRK software interrupt costs 7 CPU cycles, matching the real NMOS 6502 (2 to fetch opcode + operand, 3 to push PC and P to stack, 2 to read the IRQ vector). Undocumented opcodes are treated as BRK 0 at the same 7-cycle cost. Key differences from timer interrupts:
 - The I flag is ignored: writes always commit after BRK (the “NMI” is unconditional).
 - The operand byte b (the byte following BRK) is examined *before* registers are saved. Copy and swap operations (below) use the cell’s pre-interrupt state. This means a BRK copy produces a child that inherits the *previous* scheduling’s saved registers, not the post-BRK state.
 - The B flag (bit 4) is set in the saved P, following the 6502 convention that BRK and PHP set B while IRQ and NMI clear it. This enables fork detection: after a BRK copy, the child inherits B=0 (pre-BRK state) while the parent gets B=1. Programs can read byte 0xFB and test bit 4.
 - BRK with operand 0 resets PC to 0x0000 (preventing zero-filled cells from marching their PC into the RNG region). All other operands advance PC past the BRK+operand as usual.

- The operand byte b selects an operation: The memory-mapped neighborhood has $N^2 = 49$ cells. The 5 inner cells (origin plus 4 cardinal neighbors, indices 0–4) may be used as swap sources.

Operand b	Operation
0	Resets PC to 0x0000. Yields to scheduler.
$1 \leq b < 245$	Swap cells: source = $\lfloor b/49 \rfloor$ (cells 0–4), destination = $b \bmod 49$ (cells 0–48). Self-swaps (src = dest) update move times but do not copy data. Yields to scheduler.
$245 \leq b \leq 252$	Noisy copy: origin cell is copied to cell $b - 244$ (cells 1–8), subject to the noise model (see below). Yields to scheduler.
253–255	Reserved (no operation). Yields to scheduler.

Noisy copy model

The noisy copy operation (operands 245–252) copies all 1024 bytes of the origin cell to the destination. Each bit is independently randomized (replaced by a fair coin flip) with probability ε (**pBitNoise**), and faithfully copied with probability $1 - \varepsilon$. The default is $\varepsilon = 1/2048$, giving approximately 1 bit error per 256-byte page copied (~ 4 errors per cell).

LDA/STA writes (the normal 6502 store instructions) are always exact—the noise applies only to BRK noisy copies.

The guiding principle when considering adding more software interrupts, or expanding the OS in any way, should be to not add any functionality that can’t be justified as “physics”.

- After any copy or swap, control returns to the scheduler.
- A software interrupt due to a bad instruction is handled like BRK 0 (PC resets to 0, control returns to scheduler).

3 Randomness

Several aspects of the VM rely on randomness: the scheduling order and orientation of cells, the inter-interrupt timing, the four pseudorandom bytes written to zero-page addresses 0xFC–0xFF, and the bit-noise in the noisy copy operation.

In the reference implementation, all randomness is generated by a single deterministic pseudorandom number generator (a Mersenne Twister seeded at initialization). This makes simulations fully reproducible given the same seed, which is valuable for debugging, regression testing, and scientific reproducibility.

However, nothing in the VM specification requires deterministic randomness. Hardware or software implementations may substitute true random number generators (e.g. hardware entropy sources) for any or all of these random processes. The system’s behavior should be qualitatively the same: the “physics” of the

board—scheduling fairness, copy noise, and the statistical properties of the RNG bytes visible to programs—depend only on the *distribution* of the random variables, not on their deterministic reproducibility.

Programs should not rely on correlations between successive RNG values or between the RNG and the scheduling order. A correct program must work under any source of randomness that satisfies the specified distributions.

4 Links

- Avida (Ofria and Wilke, 2004; Ortega et al., 2023) [https://en.wikipedia.org/wiki/Avida_\(software\)](https://en.wikipedia.org/wiki/Avida_(software))
- Core War https://en.wikipedia.org/wiki/Core_War
- The BBC Micro Bot <https://mastodon.me.uk/@bbcmicrobot>

References

- Ofria, C. and Wilke, C. O. (2004). Avida: a software platform for research in computational evolutionary biology. *Artif Life*, 10(2):191–229.
- Ortega, R., Wulff, E., and Fortuna, M. A. (2023). Ontology for the Avida digital evolution platform. *Sci Data*, 10(1):608.